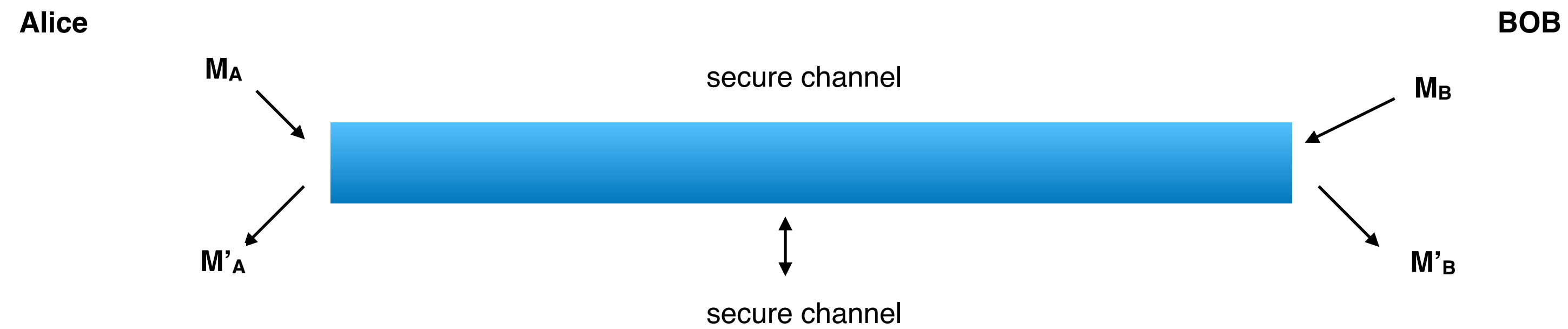


CRYPTOGRAPHY IMPLEMENTATIONS

Cryptography

The basic tasks of cryptography is establishing a secure channel. We want to do it and do it right.



Privacy: Adversary does not learn anything about M_A and M_B

Authenticity: $M'_A = M_A$ and $M'_B = M_B$

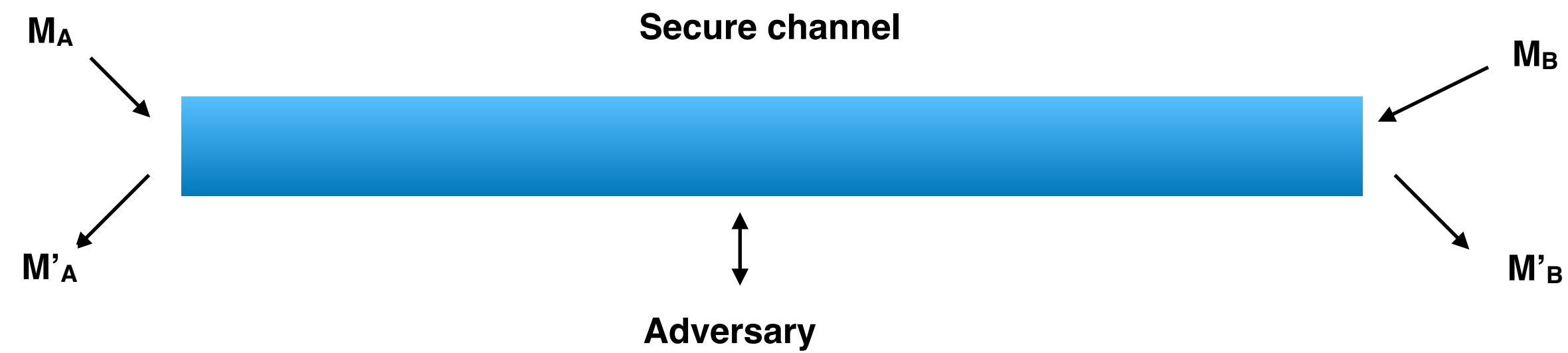
Cryptography

Who are “Alice” and “Bob”?

Alice



BOB



Cryptography

Identity

Names are bound to people by social means

Alice

Alice in wonderland



Alice Waters



Alice Walker



Alice Poker



BOB



Bob squarepants



Bob the builder



Bob Marley



Bob Marley

Cryptography

Identity

Names are bound to people by social means

Alice



Alice in wonderland

Alice Waters

Alice Walker

Alice Poker



BOB



Bob squarepants

Bob the builder

Bob Marley

Bob Marley

Established out-of-band

Biding of name to a person

Cryptography

Identity

Identity in the internet

These are difficult issues

Established how?

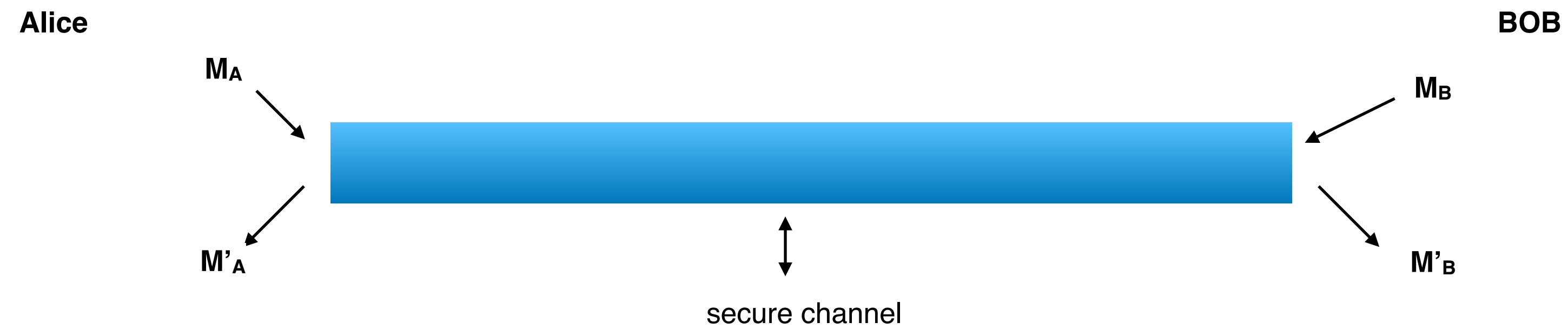
What do “domain name” and “entity” even mean?

google.com	
amazon.com	
snapchat.com	
twitter.com	

Binding of domain name to entity

Cryptography

SECURE CHANNEL TLS/SSL



Cryptography

IMPLEMENTATIONS - OpenSSL

Secure Sockets Layer (SSL) is a standard security technology for establishing an encrypted link between a server and a client. SSL allows sensitive information such as credit card numbers, social security numbers, and login credentials to be transmitted securely. Normally, data sent between browsers and web servers is sent in plain text leaving you vulnerable to eavesdropping. If an attacker is able to intercept all data being sent between a browser and a web server, they can see and use that information.

More specifically, SSL is a security protocol. Protocols describe how algorithms should be used. In this case, the SSL protocol determines variables of the encryption for both the link and the data being transmitted.

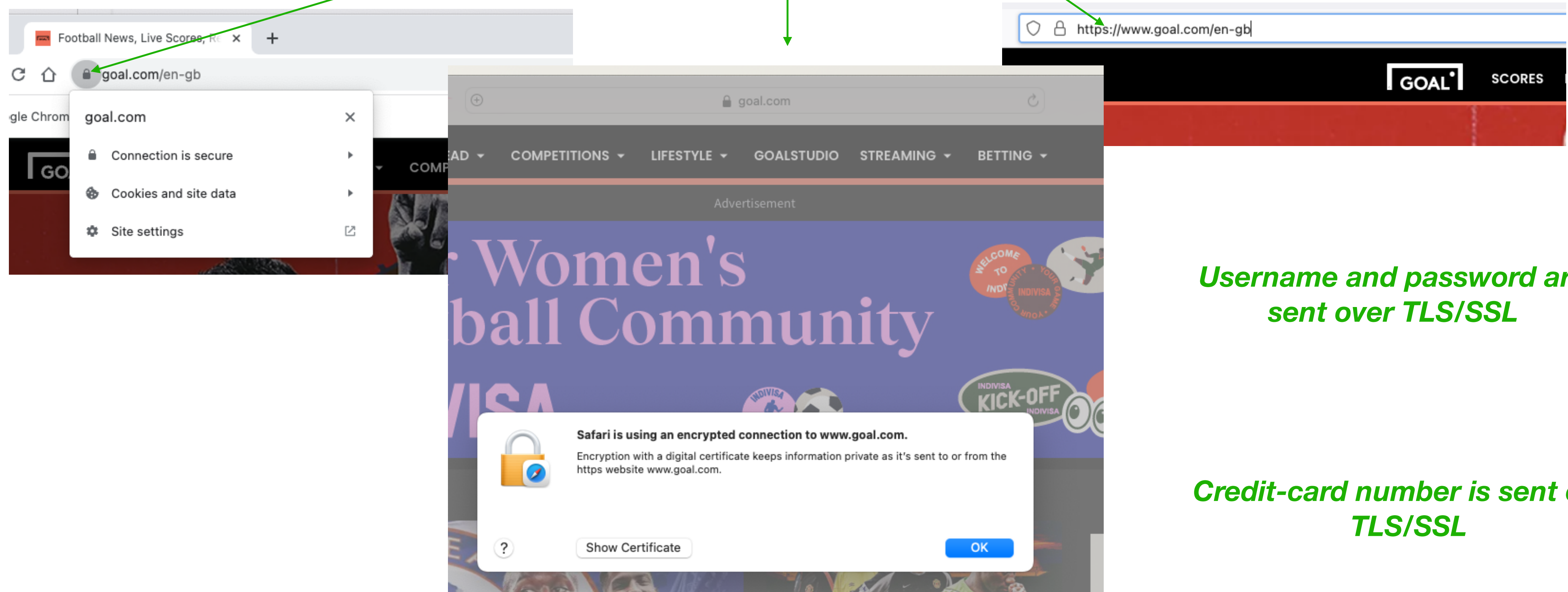
All browsers have the capability to interact with secured web servers using the SSL protocol. However, the browser and the server need what is called an SSL Certificate to be able to establish a secure connection.

OpenSSL is a software library for applications that secure communications over computer networks against eavesdropping or need to identify the party at the other end. It is widely used in internet web servers, serving a majority of all web sites.

OpenSSL contains an open-source implementation of the SSL and TLS protocols. The core library, written in the C programming language, implements basic cryptographic functions and provides various utility functions. Wrappers allowing the use of the OpenSSL library in a variety of computer languages are available.

Cryptography

SECURE CHANNEL TLS/SSL



Username and password are sent over TLS/SSL

Credit-card number is sent over TLS/SSL

Cryptography

Alice - Client

Bob - Server

***Think of Alice as a client and Bob
as a server, e.g., example.com***

google.com	
amazon.com	
snapchat.com	
twitter.com	

Cryptography

Alice - Client

I want to talk to bob.com

Hello, I am Alice

Bob - Server



Cryptography

Alice - Client

I want to talk to bob.com

Hello, I am Alice

Hello, I am Bob. Here is my
certificate to prove it.



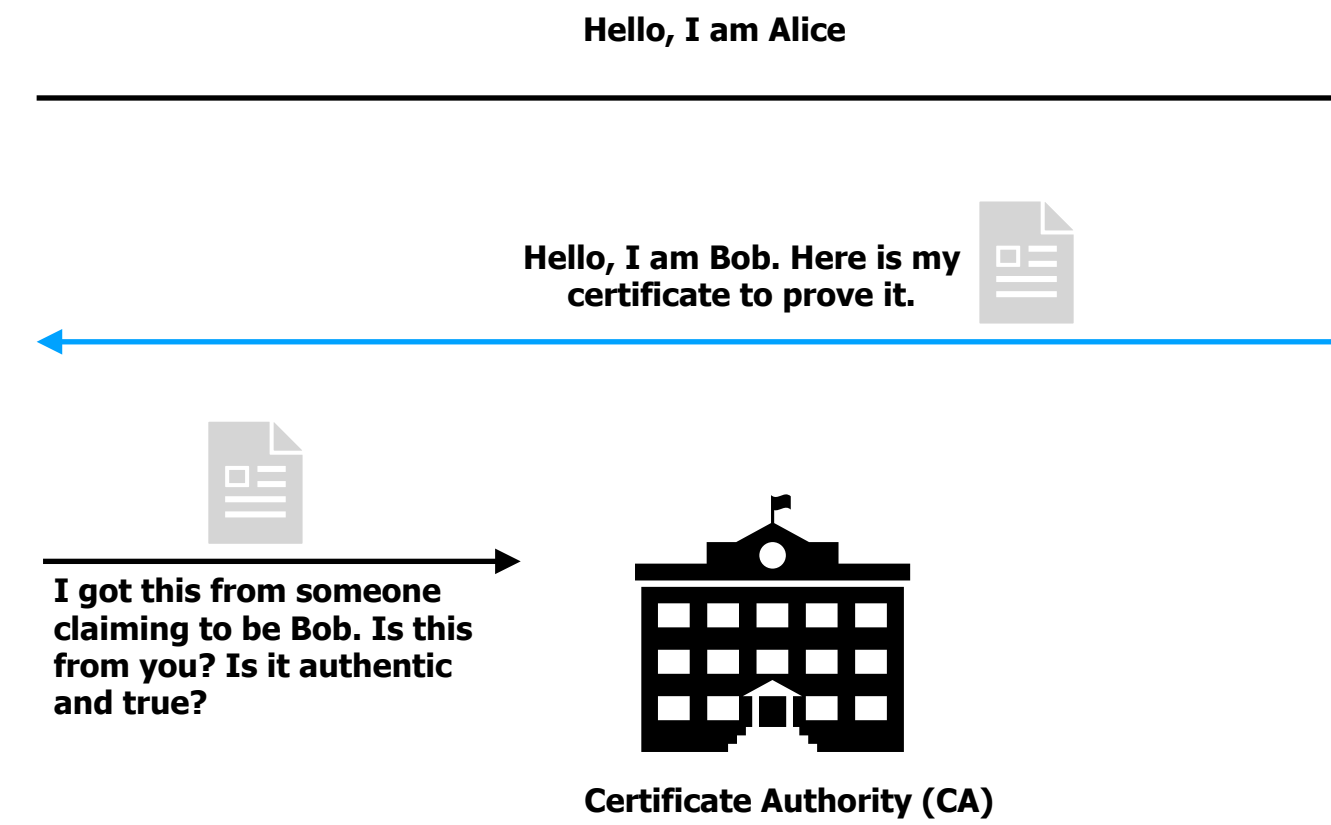
Bob - Server

Cryptography

Alice - Client

I want to talk to bob.com

Bob - Server

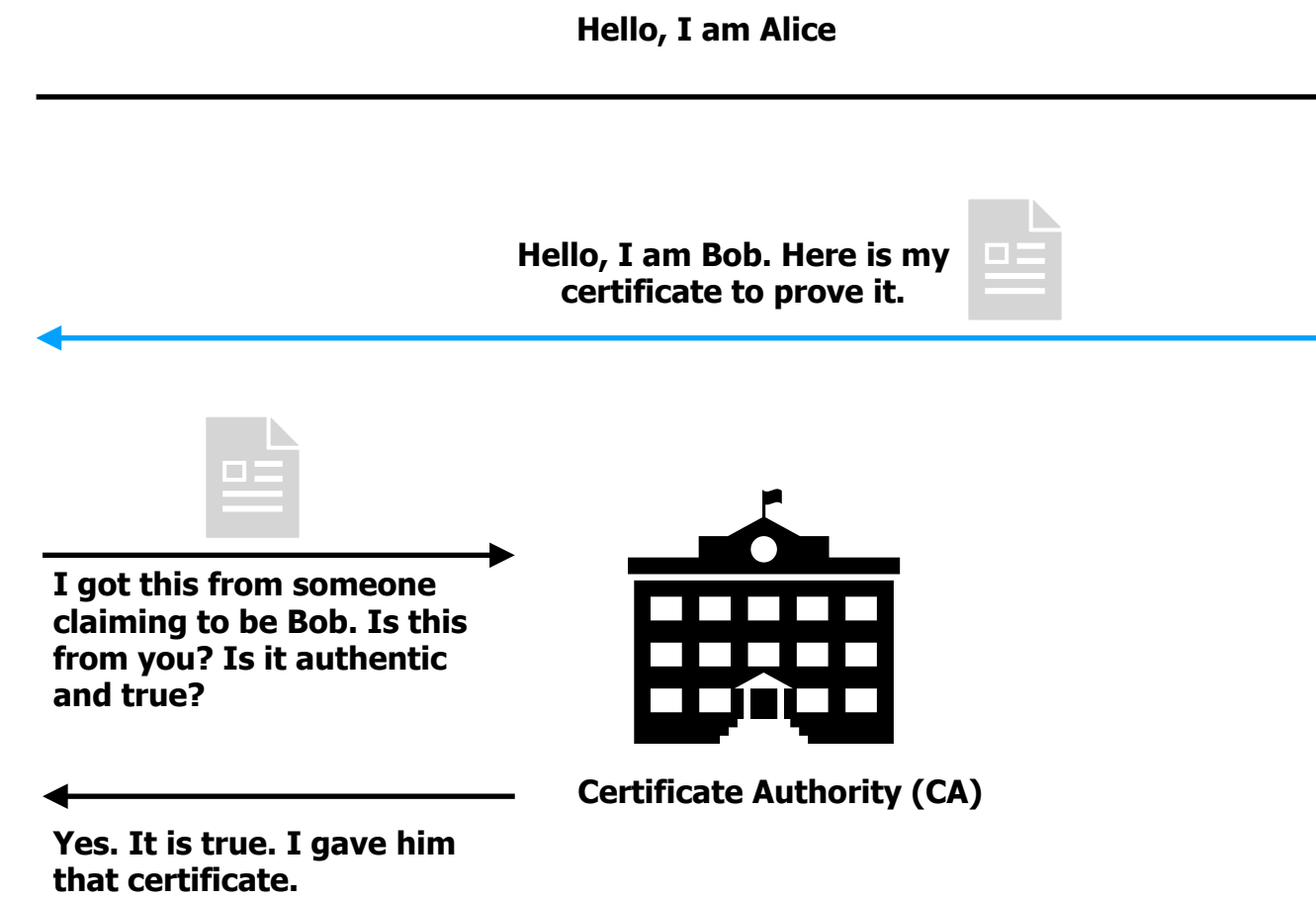


Cryptography

Alice - Client

I want to talk to bob.com

Bob - Server



Cryptography

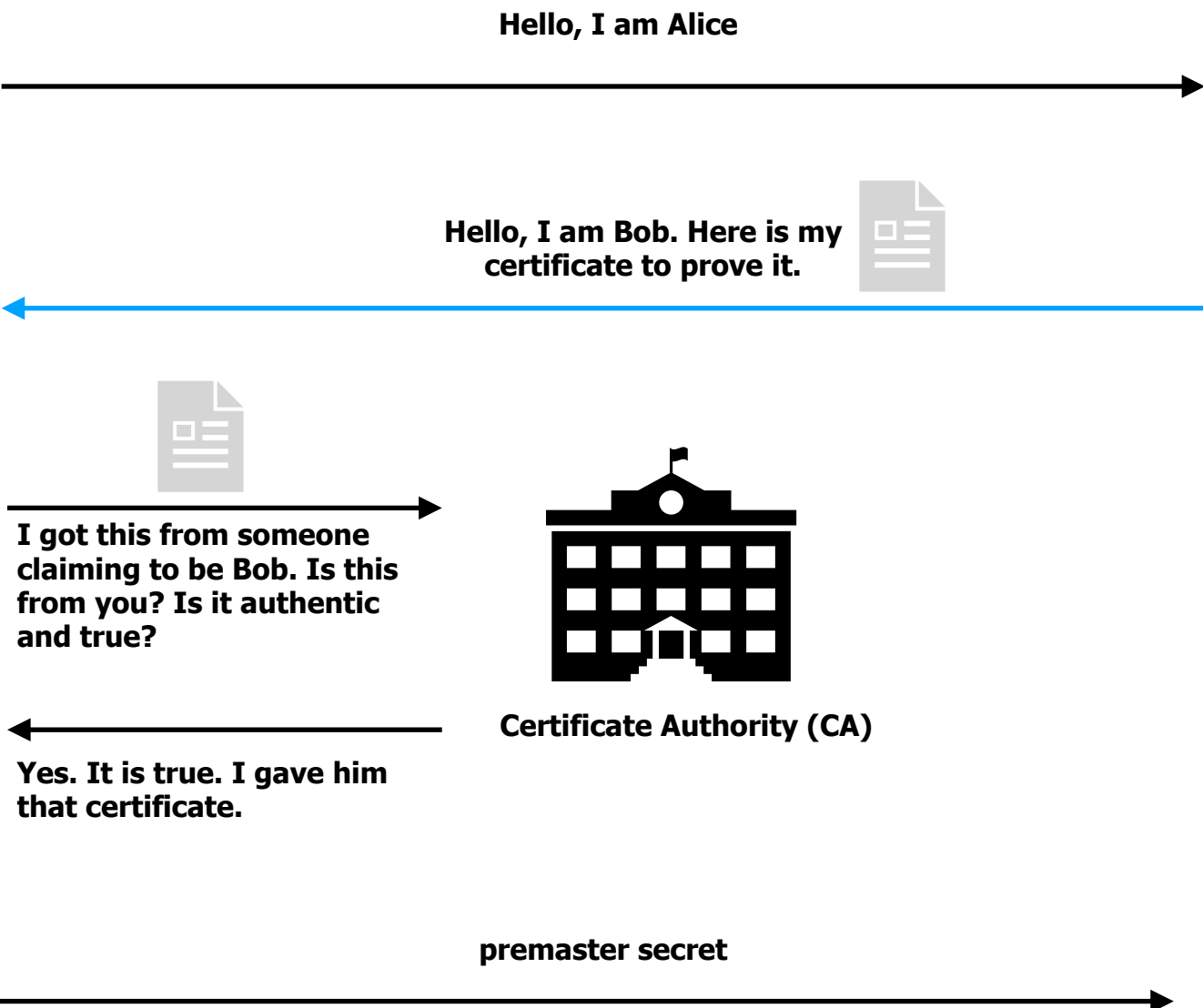
Alice - Client

I want to talk to bob.com

Bob - Server

Encrypts a secret with the public key of the server from the certificate

Decrypts the message using its private key.



Cryptography

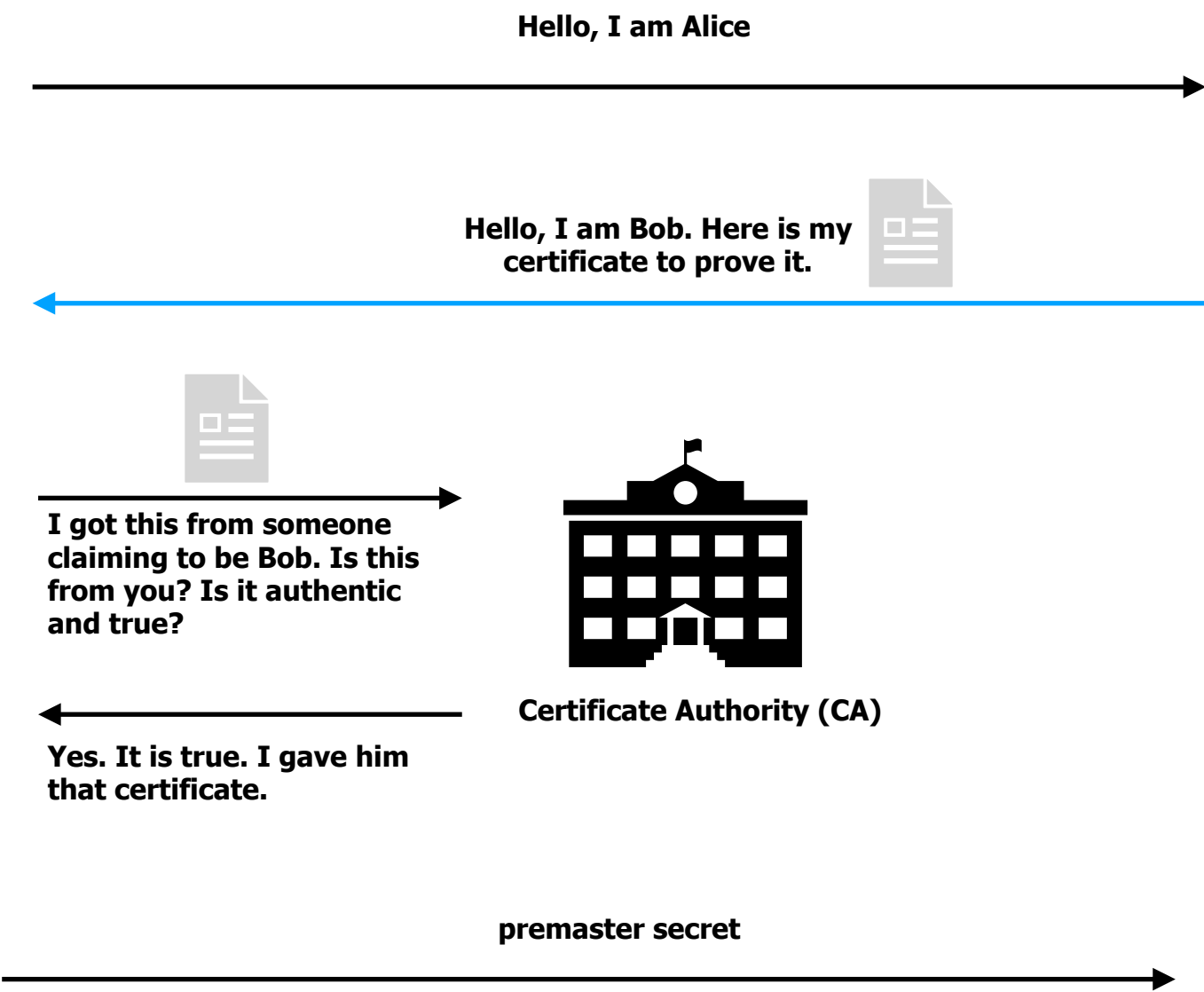
Alice - Client

I want to talk to bob.com

Bob - Server

Encrypts a secret with the public key of the server from the certificate

Calculates session keys using the premaster secret and a random value and should have the same results as the server.



Decrypts the message using its private key.

Calculates session keys using the premaster secret and a random value and should have the same results as the client.

Cryptography

Alice - Client

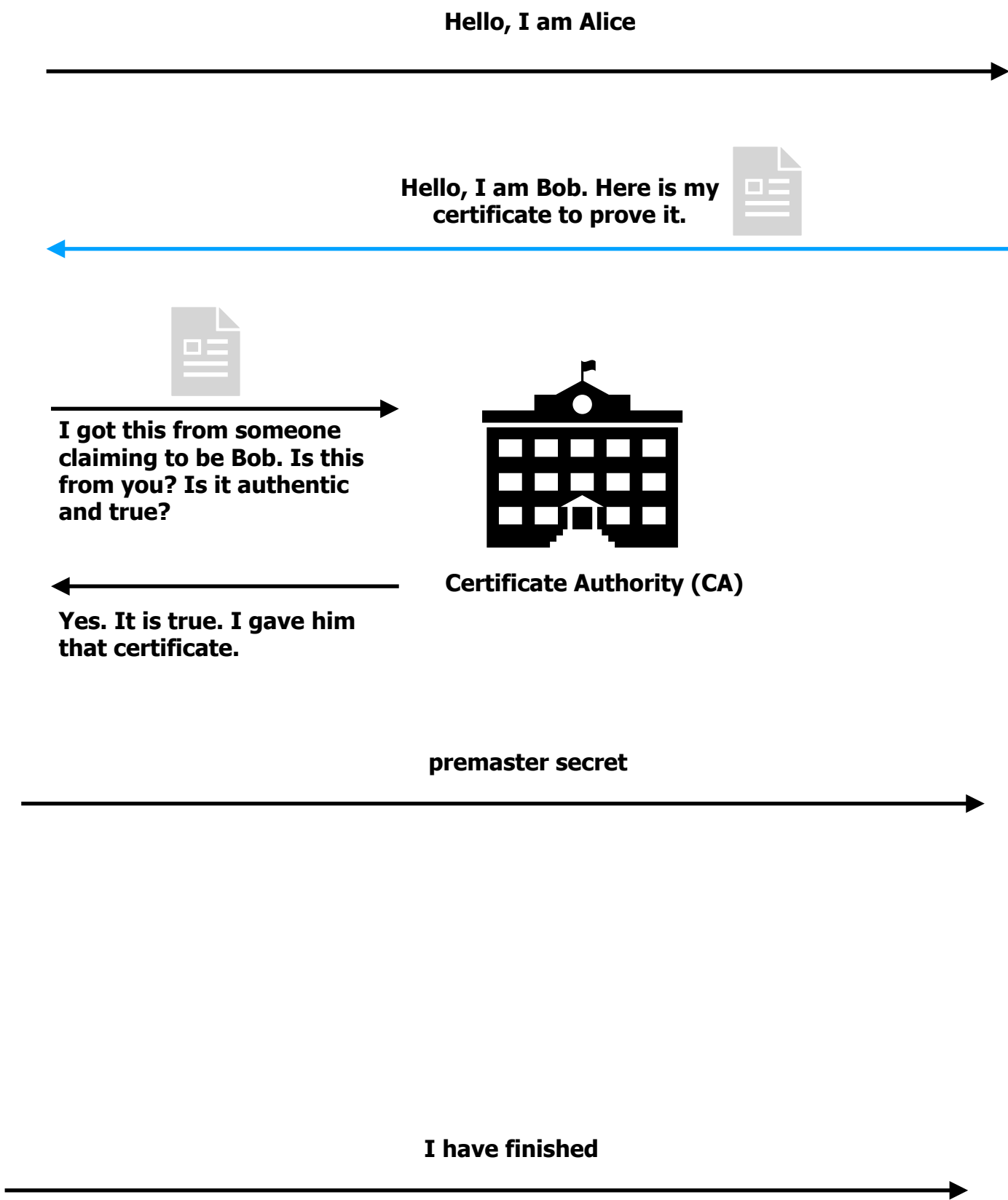
I want to talk to bob.com

Bob - Server

Encrypts a secret with the public key of the server from the certificate

Calculates session keys using the premaster secret and a random value and should have the same results as the server.

Sends "I have finished" encrypted using the session key.



Decrypts the message using its private key.

Calculates session keys using the premaster secret and a random value and should have the same results as the client.

Cryptography

Alice - Client

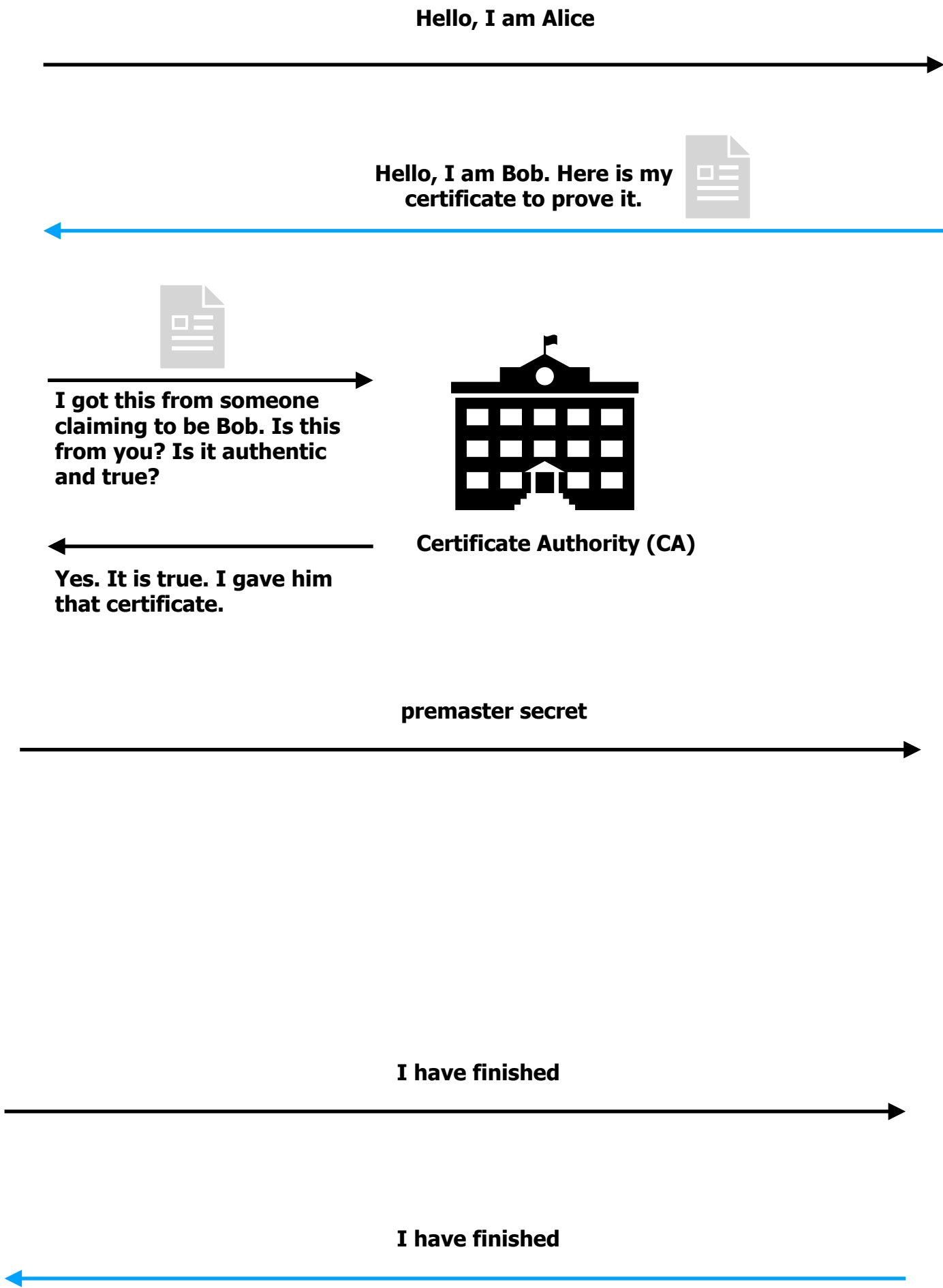
I want to talk to bob.com

Bob - Server

Encrypts a secret with the public key of the server from the certificate

Calculates session keys using the premaster secret and a random value and should have the same results as the server.

Sends "I have finished" encrypted using the session key.



Decrypts the message using its private key.

Calculates session keys using the premaster secret and a random value and should have the same results as the client.

Sends "I have finished" encrypted using the session key.

Cryptography

Alice - Client

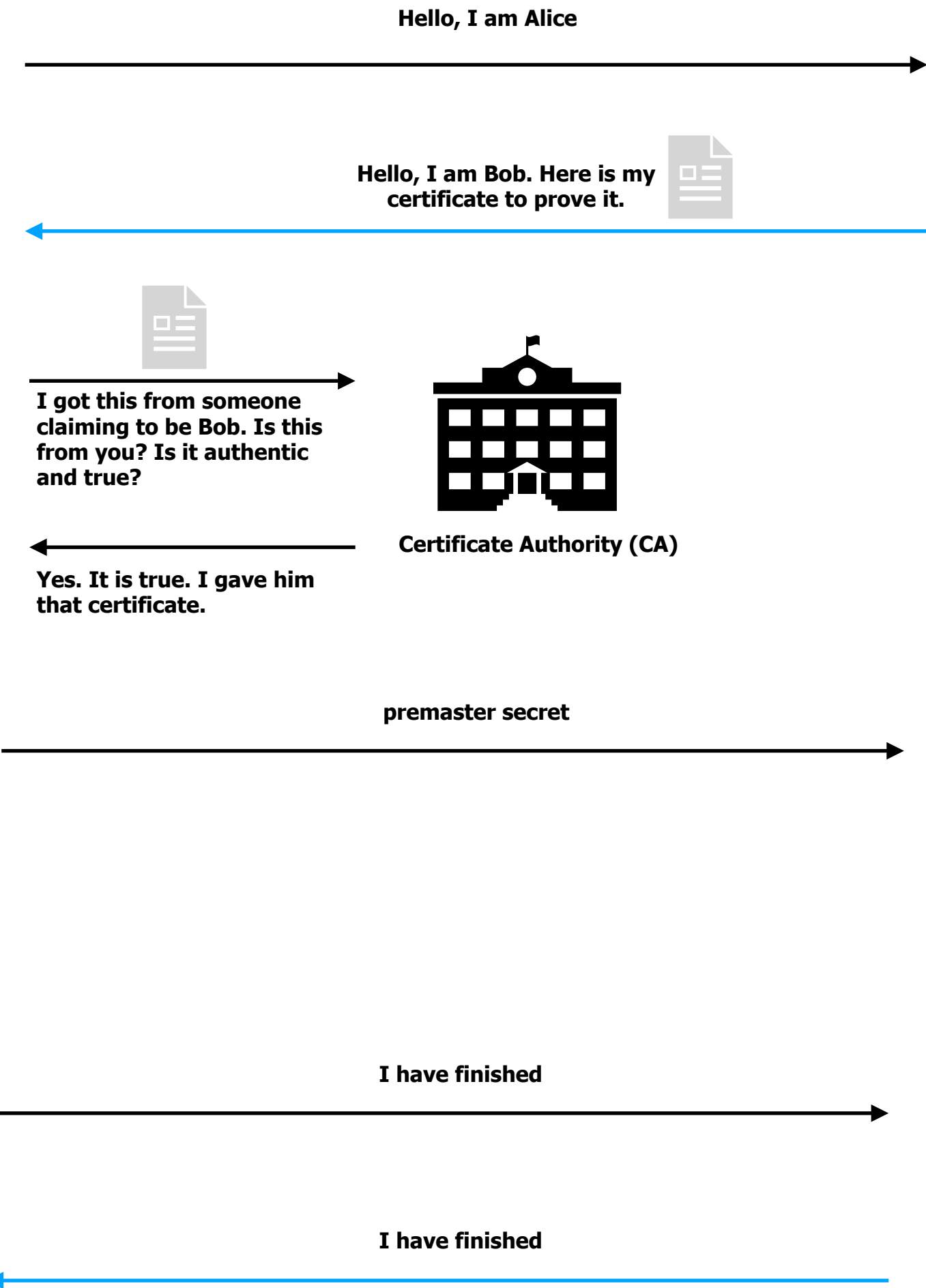
I want to talk to bob.com

Bob - Server

Encrypts a secret with the public key of the server from the certificate

Calculates session keys using the premaster secret and a random value and should have the same results as the server.

Sends "I have finished" encrypted using the session key.



Decrypts the message using its private key.

Calculates session keys using the premaster secret and a random value and should have the same results as the client.

Sends "I have finished" encrypted using the session key.

Cryptography

The exact steps within a TLS handshake will vary depending upon the kind of key exchange algorithm used and the cipher suites supported by both sides. The RSA key exchange algorithm, while now considered not secure, was used in versions of TLS before 1.3. It goes roughly as follows:

1. **The 'client hello' message:** The client initiates the handshake by sending a "hello" message to the server. The message will include which TLS version the client supports, the cipher suites supported, and a string of random bytes known as the "client random."
2. **The 'server hello' message:** In reply to the client hello message, the server sends a message containing the server's SSL certificate, the server's chosen cipher suite, and the "server random," another random string of bytes that's generated by the server.
3. **Authentication:** The client verifies the server's SSL certificate with the certificate authority that issued it. This confirms that the server is who it says it is, and that the client is interacting with the actual owner of the domain.
4. **The premaster secret:** The client sends one more random string of bytes, the "premaster secret." The premaster secret is encrypted with the public key and can only be decrypted with the private key by the server. (The client gets the public key from the server's SSL certificate.)
5. **Private key used:** The server decrypts the premaster secret.
6. **Session keys created:** Both client and server generate session keys from the client random, the server random, and the premaster secret. They should arrive at the same results.
7. **Client is ready:** The client sends a "finished" message that is encrypted with a session key.
8. **Server is ready:** The server sends a "finished" message encrypted with a session key.
9. **Secure symmetric encryption achieved:** The handshake is completed, and communication continues using the session keys.

TLS 1.3 does not support RSA, nor other cipher suites and parameters that are vulnerable to attack. It also shortens the TLS handshake, making a TLS 1.3 handshake both faster and more secure.

The basic steps of a TLS 1.3 handshake are:

- **Client hello:** The client sends a client hello message with the protocol version, the client random, and a list of cipher suites. Because support for insecure cipher suites has been removed from TLS 1.3, the number of possible cipher suites is vastly reduced. The client hello also includes the parameters that will be used for calculating the premaster secret. Essentially, the client is assuming that it knows the server's preferred key exchange method (which, due to the simplified list of cipher suites, it probably does). This cuts down the overall length of the handshake — one of the important differences between TLS 1.3 handshakes and TLS 1.0, 1.1, and 1.2 handshakes.
- **Server generates master secret:** At this point, the server has received the client random and the client's parameters and cipher suites. It already has the server random, since it can generate that on its own. Therefore, the server can create the master secret.
- **Server hello and "Finished":** The server hello includes the server's certificate, digital signature, server random, and chosen cipher suite. Because it already has the master secret, it also sends a "Finished" message.
- **Final steps and client "Finished":** Client verifies signature and certificate, generates master secret, and sends "Finished" message.
- **Secure symmetric encryption achieved**

Cryptography

IMPLEMENTATIONS - Pretty Good Privacy (PGP)

Pretty Good Privacy (PGP) is a security (encryption) program used to decrypt and encrypt email and authenticate email messages through digital signatures and file encryption.

PGP was first designed and developed in 1991 by Paul Zimmerman, a political activist. PGP software was owned and sold by a company called PGP Corporation, which was founded in 2002 then sold to Symantec in 2010.

PGP brought asymmetric encryption to the masses by providing the modern suite of cryptography: confidentiality, integrity, authentication, and nonrepudiation. It uses a “web of trust” model to authenticate digital certificates instead of relying on a central CA.

USES

- **Encrypting Emails**
- **Digital Signature Verification**
- **Encrypting Files**

Cryptography

IMPLEMENTATIONS - OpenSSH

SSH is a family of protocols that provide a secure encrypted channel connecting two networked computers over an insecure network.

It was initially designed to allow secure login between a user and a remote computer. This replaced the older *telnet*, *rlogin*, and *rsh* programs that provided unencrypted versions of this service and sent unencrypted passwords over the network.

The first version was designed by Tatu Ylonen at Helsinki University of Technology, Finland, and released as freeware in July 1995.

SSH - 2

By 2006, an improved but incompatible version 2 of the SSH protocol was adopted as a standard by the Internet Engineering Task Force (IETF).

Development of OpenSSH continues to this day, allowing more and more applications derive the benefits of secure encrypted communications.

Cryptography

IMPLEMENTATIONS - OpenSSH

SSH - Protocol

SSH is based on public key cryptography. Each machine has a public and private host key. Each user also has a public and private user key.

When logging onto a remote machine, the client first authenticates the remote host key using the public key for the host with that domain name or IP address found in the local known hosts file.

The actual protocol is much more complicated than this.

It include negotiations for which cipher to use, how to generate the shared session, what an encrypted packet looks like, and so forth.

Cryptography

IMPLEMENTATIONS - FIPS

What is FIPS 140-2 and why is it important?

FIPS (Federal Information Processing Standard) 140-2 is the benchmark for validating the effectiveness of cryptographic hardware. If a product has a FIPS 140-2 certificate you know that it has been tested and formally validated by the U.S. and Canadian Governments.

Although FIPS 140-2 is a U.S./Canadian Federal standard, FIPS 140-2 compliance has been widely adopted around the world in both governmental and non-governmental sectors as a practical security benchmark and realistic best practice.

Cryptography

IMPLEMENTATIONS - FIPS

What is FIPS 140-2 and why is it important?

Organisations use the FIPS 140-2 standard to ensure that the hardware they select meets specific security requirements. The FIPS certification standard defines four increasing, qualitative levels of security:

Level 1: Requires production-grade equipment and externally tested algorithms.

Level 2: Adds requirements for physical tamper-evidence and role-based authentication. Software implementations must run on an Operating System approved to Common Criteria at EAL2.

Level 3: Adds requirements for physical tamper-resistance and identity-based authentication. There must also be physical or logical separation between the interfaces by which “critical security parameters” enter and leave the module. Private keys can only enter or leave in encrypted form.

Level 4: This level makes the physical security requirements more stringent, requiring the ability to be tamper-active, erasing the contents of the device if it detects various forms of environmental attack.

The FIPS 140-2 standard technically allows for software-only implementations at level 3 or 4 but applies such stringent requirements that none have been validated.

Cryptography

<https://blog.cryptographyengineering.com/2022/06/09/in-defense-of-cryptocurrency/>